

```

1  import torch
2  import torch.nn as nn
3  import numpy as np
4  import matplotlib.pyplot as plt
5
6  class TrigramAttention(nn.Module):
7      """
8      Modified self-attention mechanism that implements trigram constraints.
9      Each token can only attend to itself and the previous two tokens.
10     """
11
12     def __init__(self, n_embd, n_head):
13         super().__init__()
14         self.n_embd = n_embd
15         self.n_head = n_head
16         assert n_embd % n_head == 0, "embedding dimension must be divisible by
number of heads"
17         self.head_size = n_embd // n_head
18
19         # Key, query, value projections
20         self.key = nn.Linear(n_embd, n_embd)
21         self.query = nn.Linear(n_embd, n_embd)
22         self.value = nn.Linear(n_embd, n_embd)
23
24         # Output projection
25         self.proj = nn.Linear(n_embd, n_embd)
26
27         # Register a buffer for the trigram attention mask
28         self.register_buffer("trigram_mask", None)
29
30     def forward(self, x):
31         B, T, C = x.size() # batch size, sequence length, embedding dimensionality
32
33         # If trigram mask doesn't exist or has wrong size, create it
34         if self.trigram_mask is None or self.trigram_mask.shape[1] != T:
35             # Create trigram mask (each position can only attend to itself and 2 previous
positions)
36             trigram_mask = torch.tril(torch.ones(T, T), diagonal=0)
37
38             # Zero out attention to positions beyond the trigram window
39             for i in range(T):
40                 for j in range(T):
41                     if i - j > 2: # If position j is more than 2 steps before position i
42                         trigram_mask[i, j] = 0
43
44             self.register_buffer("trigram_mask", trigram_mask.view(1, 1, T, T))
45
46         # Linear projections
47         k = self.key(x).view(B, T, self.n_head, self.head_size).transpose(1, 2) # (B, nh, T
, hs)
48         q = self.query(x).view(B, T, self.n_head, self.head_size).transpose(1, 2) # (B, nh
, T, hs)
49         v = self.value(x).view(B, T, self.n_head, self.head_size).transpose(1, 2) # (B, nh,

```

```

49     T, hs)
50
51     # Compute attention scores
52     att = (q @ k.transpose(-2, -1)) * (1.0 / np.sqrt(k.size(-1))) # (B, nh, T, T)
53
54     # Apply trigram attention mask
55     att = att.masked_fill(self.trigram_mask == 0, float('-inf'))
56
57     # Softmax attention weights
58     att = torch.softmax(att, dim=-1)
59
60     # Apply attention to values
61     y = att @ v # (B, nh, T, hs)
62     y = y.transpose(1, 2).contiguous().view(B, T, C) # (B, T, C)
63
64     # Output projection
65     y = self.proj(y)
66
67     return y
68
69
70 class TrigramTransformerBlock(nn.Module):
71     """Transformer block modified to implement trigram constraints"""
72
73     def __init__(self, n_embd, n_head, dropout=0.1):
74         super().__init__()
75         # Use trigram attention instead of standard causal attention
76         self.attn = TrigramAttention(n_embd, n_head)
77         self.ln1 = nn.LayerNorm(n_embd)
78         self.ln2 = nn.LayerNorm(n_embd)
79         self.mlp = nn.Sequential(
80             nn.Linear(n_embd, 4 * n_embd),
81             nn.GELU(),
82             nn.Linear(4 * n_embd, n_embd),
83             nn.Dropout(dropout),
84         )
85         self.dropout = nn.Dropout(dropout)
86
87     def forward(self, x):
88         x = x + self.dropout(self.attn(self.ln1(x)))
89         x = x + self.dropout(self.mlp(self.ln2(x)))
90         return x
91
92
93 class TrigramGPT(nn.Module):
94     """GPT-like model restricted to the trigram context window"""
95
96     def __init__(
97         self,
98         vocab_size,
99         n_embd=128,
100         n_head=4,
101         n_layer=4,

```

```

102     max_seq_len=256,
103     dropout=0.1
104 ):
105     super().__init__()
106     self.vocab_size = vocab_size
107     self.n_embd = n_embd
108     self.max_seq_len = max_seq_len
109
110     # Token and positional embeddings
111     self.tok_emb = nn.Embedding(vocab_size, n_embd)
112     self.pos_emb = nn.Parameter(torch.zeros(1, max_seq_len, n_embd))
113
114     # Transformer blocks with trigram attention
115     self.transformer_blocks = nn.ModuleList([
116         TrigramTransformerBlock(n_embd, n_head, dropout) for _ in range(n_layer)
117     ])
118
119     self.ln_f = nn.LayerNorm(n_embd)
120     self.lm_head = nn.Linear(n_embd, vocab_size, bias=False)
121     self.dropout = nn.Dropout(dropout)
122
123     # Initialize weights
124     self.apply(self._init_weights)
125
126     def _init_weights(self, module):
127         if isinstance(module, nn.Linear):
128             torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
129             if module.bias is not None:
130                 torch.nn.init.zeros_(module.bias)
131         elif isinstance(module, nn.Embedding):
132             torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
133
134     def forward(self, idx, targets=None):
135         B, T = idx.size()
136         assert T <= self.max_seq_len, "Sequence length exceeds model capacity"
137
138         # Get token embeddings
139         token_embeddings = self.tok_emb(idx) # (B, T, C)
140
141         # Add positional embeddings
142         position_embeddings = self.pos_emb[:, :T, :] # (1, T, C)
143         x = self.dropout(token_embeddings + position_embeddings) # (B, T, C)
144
145         # Apply transformer blocks
146         for block in self.transformer_blocks:
147             x = block(x)
148
149         # Final layer norm and output projection
150         x = self.ln_f(x)
151         logits = self.lm_head(x) # (B, T, vocab_size)
152
153         # Compute loss if targets are provided
154         loss = None

```

```

155     if targets is not None:
156         # Reshape logits and compute cross entropy loss
157         B, T, C = logits.size()
158         logits = logits.view(B * T, C)
159         targets = targets.view(B * T)
160         loss = nn.functional.cross_entropy(logits, targets)
161
162     return logits, loss
163
164     def generate(self, idx, max_new_tokens, temperature=1.0):
165         """Generate new tokens autoregressively"""
166         for _ in range(max_new_tokens):
167             # Get last three tokens (trigram context)
168             idx_cond = idx[:, -3:] if idx.size(1) > 3 else idx
169
170             # Forward pass to get logits
171             with torch.no_grad():
172                 logits, _ = self.forward(idx_cond)
173
174             # Get logits for the last token
175             logits = logits[:, -1, :] / temperature
176
177             # Apply softmax to get probabilities
178             probs = nn.functional.softmax(logits, dim=-1)
179
180             # Sample next token
181             idx_next = torch.multinomial(probs, num_samples=1)
182
183             # Append to the sequence
184             idx = torch.cat([idx, idx_next], dim=1)
185
186         return idx
187
188     # Example usage and visualization
189     def visualize_attention_mask():
190         """Visualize the difference between standard causal mask and trigram mask"""
191         seq_len = 10
192
193         # Standard causal mask (lower triangular)
194         causal_mask = torch.tril(torch.ones(seq_len, seq_len))
195
196         # Trigram mask
197         trigram_mask = torch.tril(torch.ones(seq_len, seq_len))
198         for i in range(seq_len):
199             for j in range(seq_len):
200                 if i - j > 2: # If position j is more than 2 steps before position i
201                     trigram_mask[i, j] = 0
202
203         # Visualize the masks
204         fig, axes = plt.subplots(1, 2, figsize=(12, 5))
205
206         axes[0].matshow(causal_mask, cmap='viridis')
207

```

```

208 axes[0].set_title('Standard Causal Mask')
209 axes[0].set_xlabel('Key position')
210 axes[0].set_ylabel('Query position')
211
212 axes[1].matshow(trigram_mask, cmap='viridis')
213 axes[1].set_title('Trigram Mask')
214 axes[1].set_xlabel('Key position')
215 axes[1].set_ylabel('Query position')
216
217 plt.tight_layout()
218 plt.savefig('attention_masks_comparison.png')
219
220 return fig
221
222
223 def train_and_evaluate_trigram_model():
224     """Demo function to train and evaluate the trigram model on a toy task"""
225     # Toy vocabulary and dataset for demonstration
226     vocab_size = 100
227     seq_length = 20
228     batch_size = 32
229
230     # Create a toy dataset (random sequences)
231     def get_batch():
232         # Generate random token indices
233         x = torch.randint(1, vocab_size, (batch_size, seq_length))
234         y = torch.roll(x, shifts=-1, dims=1)
235         y[:, -1] = 0 # padding
236         return x, y
237
238     # Initialize the model
239     model = TrigramGPT(vocab_size=vocab_size, max_seq_len=seq_length)
240     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
241     model.to(device)
242
243     # Training parameters
244     optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)
245     num_epochs = 10
246
247     # Training loop
248     losses = []
249     for epoch in range(num_epochs):
250         epoch_loss = 0
251         num_batches = 100
252
253         for _ in range(num_batches):
254             x, y = get_batch()
255             x, y = x.to(device), y.to(device)
256
257             # Forward pass
258             logits, loss = model(x, y)
259
260             # Backward pass

```

```

261     optimizer.zero_grad()
262     loss.backward()
263     optimizer.step()
264
265     epoch_loss += loss.item()
266
267     avg_loss = epoch_loss / num_batches
268     losses.append(avg_loss)
269     print(f'Epoch {epoch + 1}/{num_epochs}, Loss: {avg_loss:.4f}')
270
271     # Plot training loss
272     plt.figure(figsize=(10, 5))
273     plt.plot(range(1, num_epochs + 1), losses)
274     plt.xlabel('Epoch')
275     plt.ylabel('Loss')
276     plt.title('Training Loss')
277     plt.grid(True)
278     plt.savefig('trigram_model_training_loss.png')
279
280     # Generate text using the model
281     print("\nText generation example:")
282
283     start_tokens = torch.tensor([[1, 2, 3]]).to(device) # Start with some seed tokens
284     generated = model.generate(start_tokens, max_new_tokens=20)
285
286     print("Generated sequence:")
287     print(generated.cpu().numpy())
288
289     return model
290
291
292 if __name__ == "__main__":
293     print("Visualizing attention masks to compare standard causal attention vs  

trigram attention...")
294     visualize_attention_mask()
295
296     print("\nTraining and evaluating trigram language model...")
297     model = train_and_evaluate_trigram_model()
298
299     print("\nDifferences between standard transformer and trigram transformer  

:")
300     print("1. In standard transformer: each token can attend to all previous  

tokens")
301     print("2. In trigram transformer: each token can only attend to itself and  

previous two tokens")
302     print("3. Implementation difference: modified attention mask restricts the  

context window")
303
304     """
305
306     Visualizing attention masks to compare standard causal attention vs trigram  

attention...
307

```

```
308 Training and evaluating trigram language model...
309 Epoch 1/10, Loss: 4.4566
310 Epoch 2/10, Loss: 4.3779
311 Epoch 3/10, Loss: 4.3831
312 Epoch 4/10, Loss: 4.3725
313 Epoch 5/10, Loss: 4.3708
314 Epoch 6/10, Loss: 4.3708
315 Epoch 7/10, Loss: 4.3699
316 Epoch 8/10, Loss: 4.3696
317 Epoch 9/10, Loss: 4.3695
318 Epoch 10/10, Loss: 4.3693
319
320 Text generation example:
321 Generated sequence:
322 [[ 1  2  3 17 56 96 73 22 46 61 71  4 40 89 84 72 64 29  2 83 18 10 45]]
323
324 Differences between standard transformer and trigram transformer:
325 1. In standard transformer: each token can attend to all previous tokens
326 2. In trigram transformer: each token can only attend to itself and previous two
tokens
327 3. Implementation difference: modified attention mask restricts the context
window
328
329 """"
```